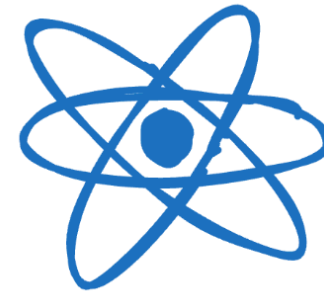
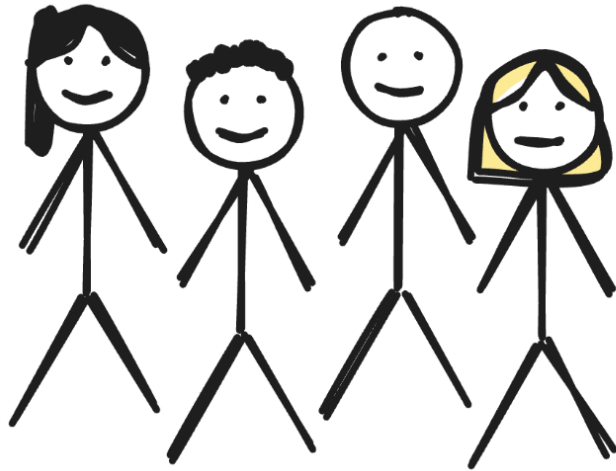


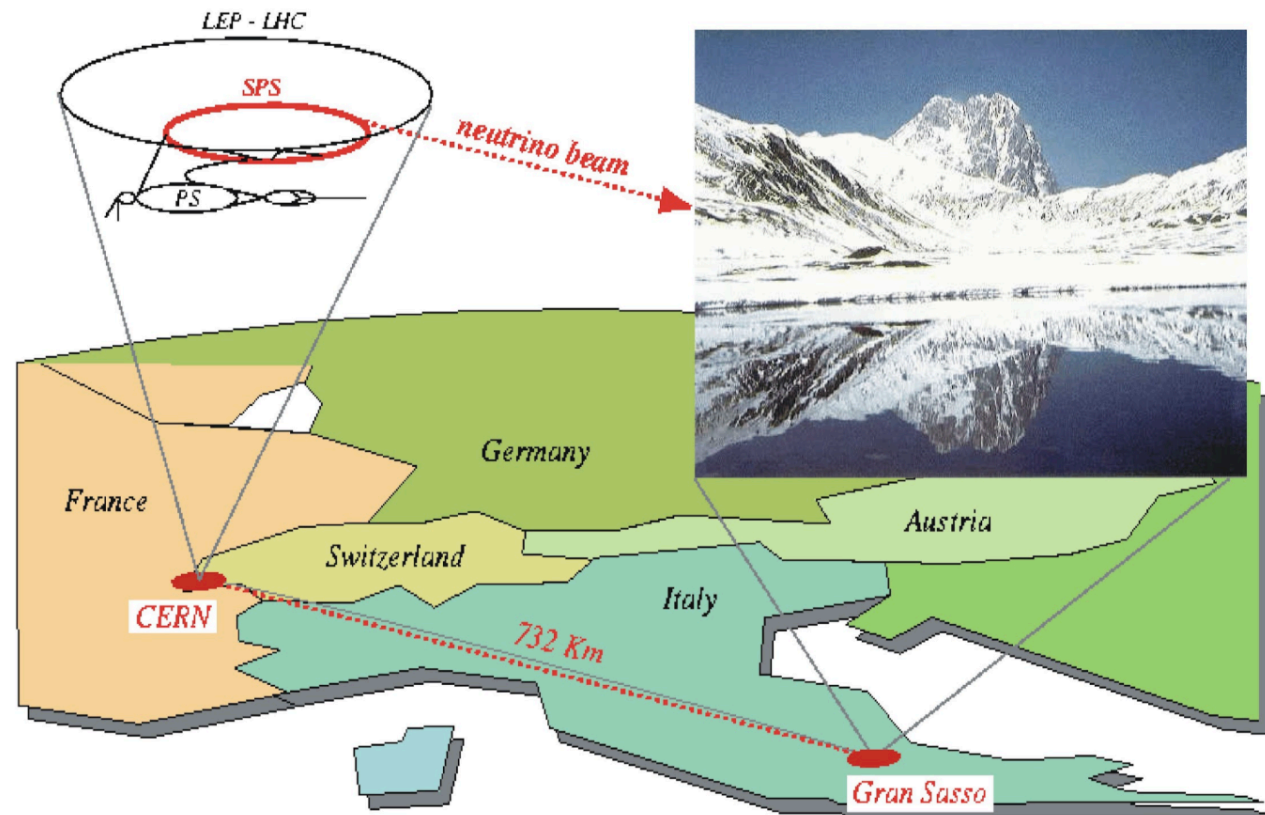
# How to Reliably Measure Software Performance

Augusto de Oliveira, Kemal Akkoyun

FOSDEM 2026



## *CERN to Gran Sasso Neutrino Beam*



5 years

100 M€

[2, 3]

# nature

---

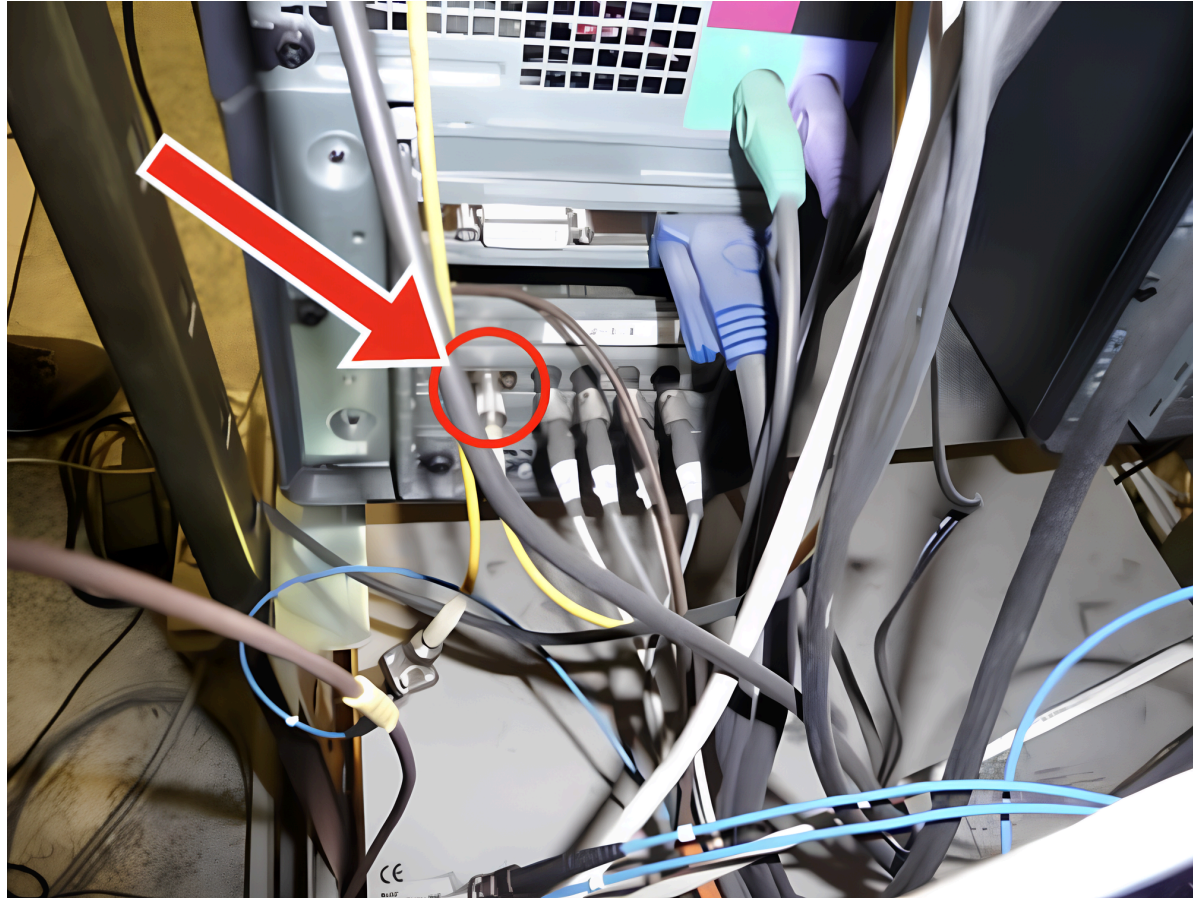
News | Published: 22 September 2011

## Particles break light-speed limit

[Geoff Brumfiel](#)

[Nature](#) (2011) | [Cite this article](#)

**2879** Accesses | **13** Citations | **854** Altmetric | [Metrics](#)



*Loose fiber optic cable that caused the measurement error [4]*

# How to Control Your Benchmarking Environment

How to:

1. Control your benchmarking environment.
2. Design your benchmarks.
3. Interpret benchmark results.
4. Integrate benchmarks into your workflows.

| Layer       | Sources of Noise                                                                | Mitigations                                                                |
|-------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| External    | Network<br>Temperature<br>Vibration<br>Noisy neighbors                          | Use dedicated on-prem hardware<br>Use dedicated/bare metal cloud instances |
| Application | Memory layout<br>Compilation/linking                                            | Set up fixed builds (e.g., disable ASLR)                                   |
| Kernel      | Filesystem cache<br>Scheduling                                                  | Set CPU affinity<br>Set process priority<br>Warm up or drop caches         |
| CPU         | Simultaneous multithreading (SMT) contention<br>Dynamic frequency scaling (DFS) | Disable SMT<br>Disable DFS                                                 |

| Layer       | Sources of Noise                                                                | Mitigations                                                                |
|-------------|---------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| External    | Network<br>Temperature<br>Vibration<br>Noisy neighbors                          | Use dedicated on-prem hardware<br>Use dedicated/bare metal cloud instances |
| Application | Memory layout<br>Compilation/linking                                            | Set up fixed builds (e.g., disable ASLR)                                   |
| Kernel      | Filesystem cache<br>Scheduling                                                  | Set CPU affinity<br>Set process priority<br>Warm up or drop caches         |
| CPU         | Simultaneous multithreading (SMT) contention<br>Dynamic frequency scaling (DFS) | Disable SMT<br>Disable DFS                                                 |

*Bakhvalov, "Performance Analysis and Tuning on Modern CPUs", Appendix A [5]*

| Layer  | Sources of Noise               | Mitigations                                                        |
|--------|--------------------------------|--------------------------------------------------------------------|
| Kernel | Filesystem cache<br>Scheduling | Set CPU affinity<br>Set process priority<br>Warm up or drop caches |

```
# Set CPU affinity
taskset -c 0 ./benchmark

# Set process priority
nice -n -5 ./benchmark

# Drop filesystem cache
echo 3 > /proc/sys/vm/drop_caches && sync
```

| Layer | Sources of Noise                                  | Mitigations                |
|-------|---------------------------------------------------|----------------------------|
| CPU   | SMT contention<br>Dynamic frequency scaling (DFS) | Disable SMT<br>Disable DFS |

```
# Disable SMT
```

```
echo off > /sys/devices/system/cpu/smt/control
```

```
# Disable DFS
```

```
echo 2500000 > /sys/devices/system/cpu/cpu*/cpufreq/scaling_min_freq
```

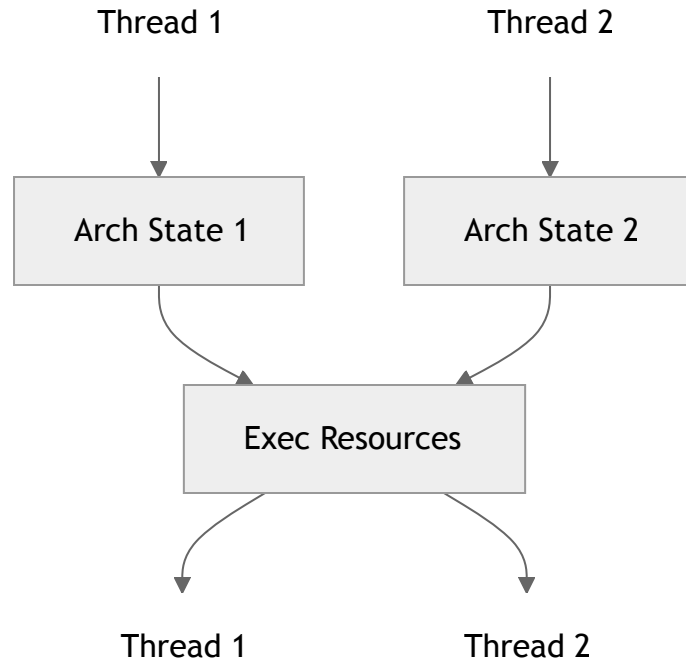
```
echo 2500000 > /sys/devices/system/cpu/cpu*/cpufreq/scaling_max_freq
```

```
echo performance > /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor
```

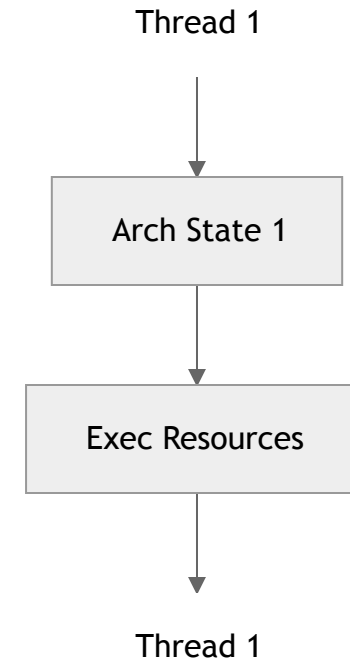
```
# Disable Turbo-Boost, Intel CPUs only
```

```
echo 1 > /sys/devices/system/cpu/intel_pstate/no_turbo
```

## What's the impact of disabling SMT?



*SMT enabled: hardware threads compete for resources*

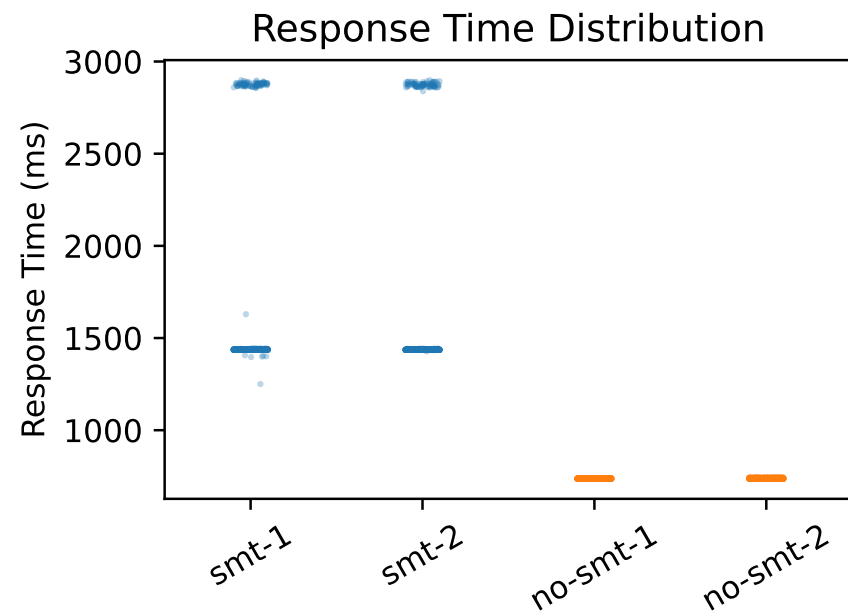


*SMT disabled: hardware thread has exclusive access to resources*

## What's the impact of disabling SMT?

m5.metal, clock rate pinned, scaling governor set to "performance"

**2 CPU-intensive tasks on same core (smt) vs. separate cores (no-smt)**

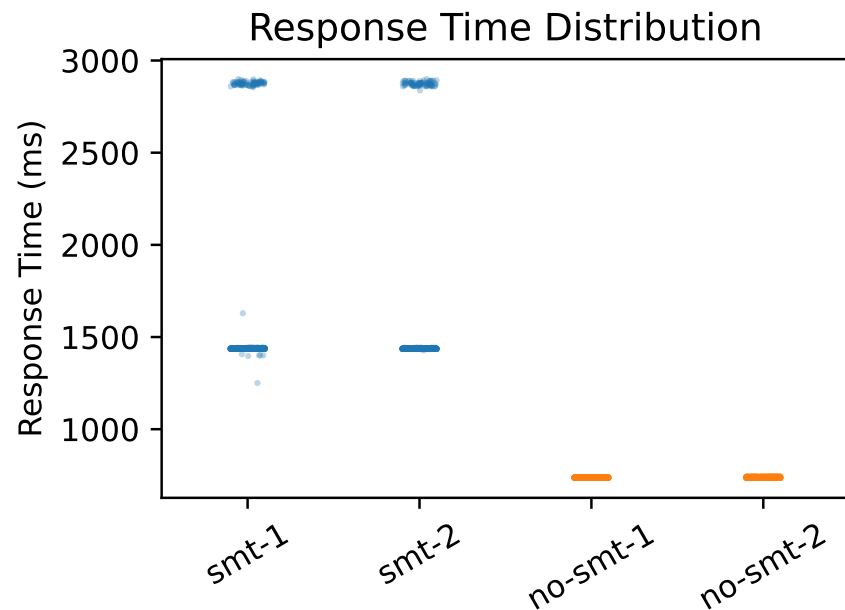


| Thread   | mean ± stddev       | coeff. of variation |
|----------|---------------------|---------------------|
| smt-1    | 1537.64 ± 367.29 ms | 23.887 %            |
| smt-2    | 1536.88 ± 366.84 ms | 23.869 %            |
| no-smt-1 | 737.37 ± 0.32 ms    | 0.044 %             |
| no-smt-2 | 737.93 ± 1.74 ms    | 0.235 %             |

## What's the impact of disabling SMT?

m5.metal, clock rate pinned, scaling governor set to "performance"

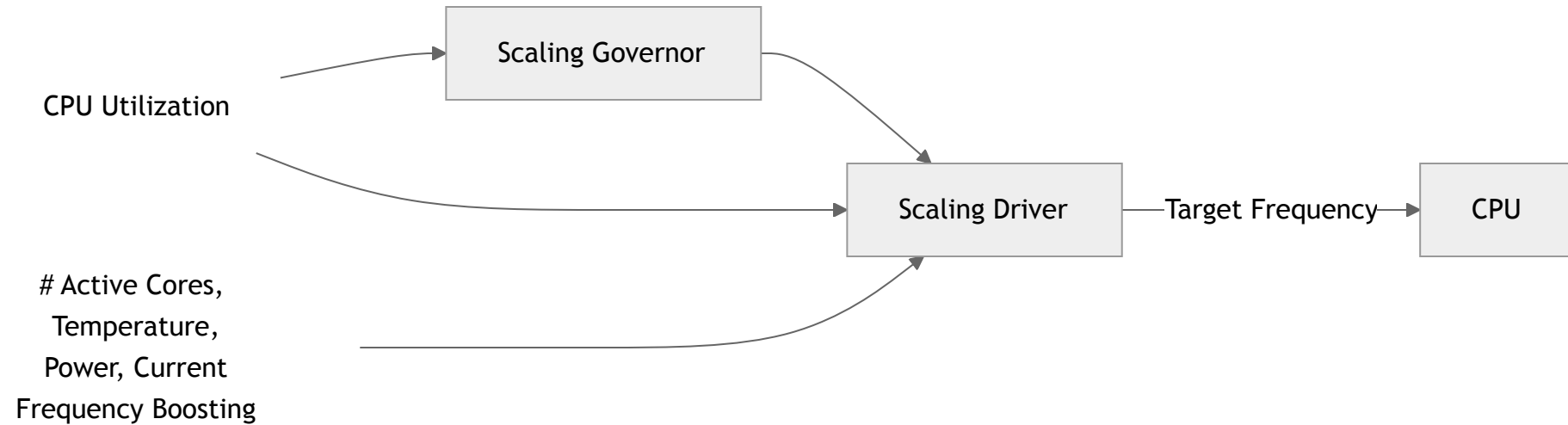
**2 CPU-intensive tasks on same core (smt) vs. separate cores (no-smt)**



| Thread   | mean ± stddev       | coeff. of variation |
|----------|---------------------|---------------------|
| smt-1    | 1537.64 ± 367.29 ms | 23.887 %            |
| smt-2    | 1536.88 ± 366.84 ms | 23.869 %            |
| no-smt-1 | 737.37 ± 0.32 ms    | 0.044 %             |
| no-smt-2 | 737.93 ± 1.74 ms    | 0.235 %             |

**100x less variation**

## What's the impact of disabling DFS?

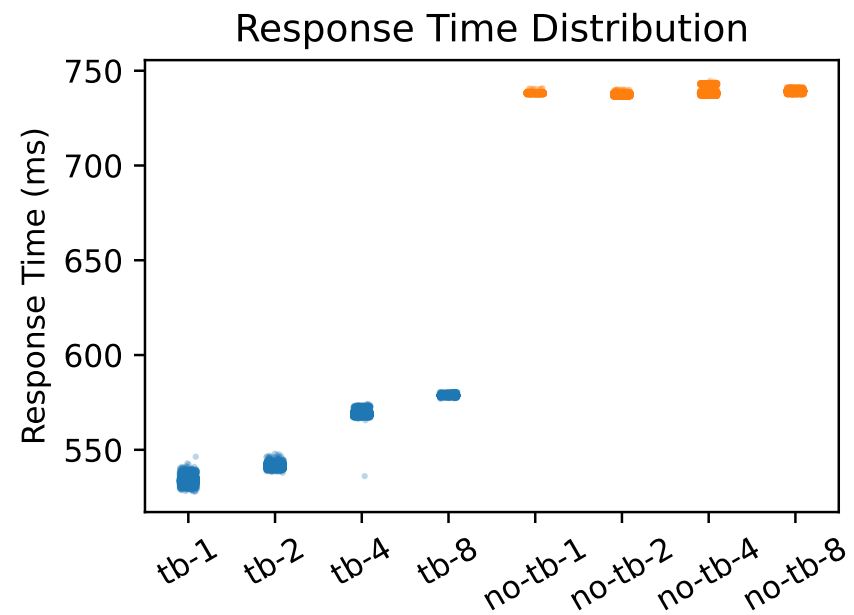


*DFS enabled: CPU frequency is automatically set by the Scaling Governor and the Scaling Driver*

# What's the impact of disabling DFS?

m5.metal, SMT disabled

Varying number of CPU-intensive tasks on the same core with DFS on vs. off

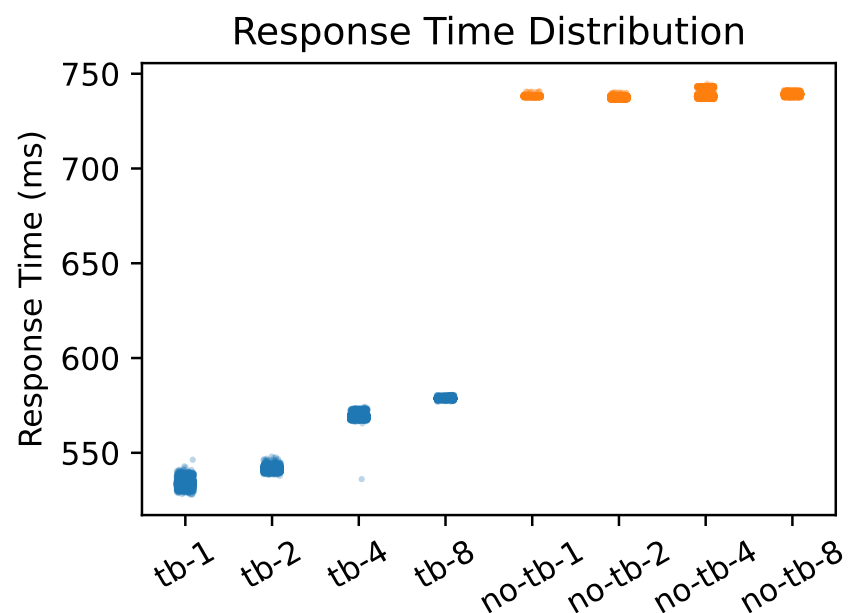


| Thread   | mean $\pm$ stddev     | coeff. of variation |
|----------|-----------------------|---------------------|
| dfs-1    | 533.97 $\pm$ 2.046 ms | 0.383 %             |
| dfs-8    | 578.67 $\pm$ 0.287 ms | 0.050 %             |
| no-dfs-1 | 738.18 $\pm$ 0.306 ms | 0.041 %             |
| no-dfs-8 | 739.18 $\pm$ 0.351 ms | 0.047 %             |

## What's the impact of disabling DFS?

m5.metal, SMT disabled

Varying number of CPU-intensive tasks on the same core with DFS on vs. off



| Thread   | mean $\pm$ stddev     | coeff. of variation |
|----------|-----------------------|---------------------|
| dfs-1    | 533.97 $\pm$ 2.046 ms | 0.383 %             |
| dfs-8    | 578.67 $\pm$ 0.287 ms | 0.050 %             |
| no-dfs-1 | 738.18 $\pm$ 0.306 ms | 0.041 %             |
| no-dfs-8 | 739.18 $\pm$ 0.351 ms | 0.047 %             |

10x less variation

Removes dynamic frequency scaling as a source of noise

| Layer    | Sources of Noise | Mitigations                   |
|----------|------------------|-------------------------------|
| External | Vibration        | Don't shout in the datacenter |



## Shouting in the Datacenter

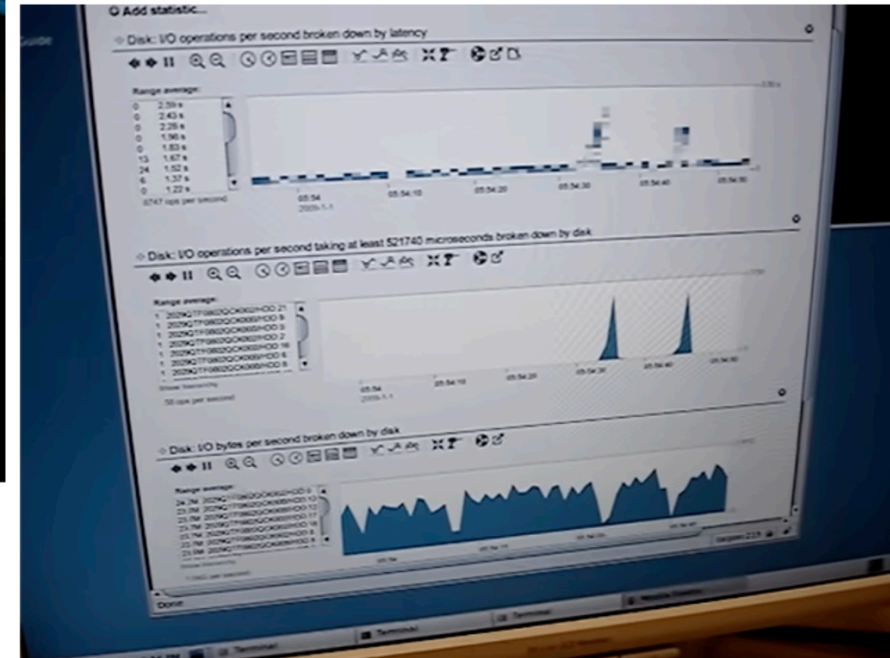


**Bryan Cantrill**  
8.08K subscribers

Subscribe

4.8M views 17 years ago

Brendan Gregg from Sun's Fishworks team makes an interesting discovery about inducing disk latency. For a ca. 2020 retrospective on this 2008 video: [YouTube](#) • Bryan Cantrill talks Sun Microsystems, DTr....more



*Shouting in the Datacenter*

# How to Design Benchmarks

*"All happy families are alike; each unhappy family is unhappy in its own way."*

— Leo Tolstoy, *Anna Karenina*

*"All happy ~~families~~ are alike; each unhappy ~~family~~ is unhappy in its own way."*  
**benchmarks benchmark**

**representative and  
repeatable**

## **An unhappy benchmark**

- Goal: Measuring dd-trace-java instrumentation overhead on a Spring app.

## An unhappy benchmark

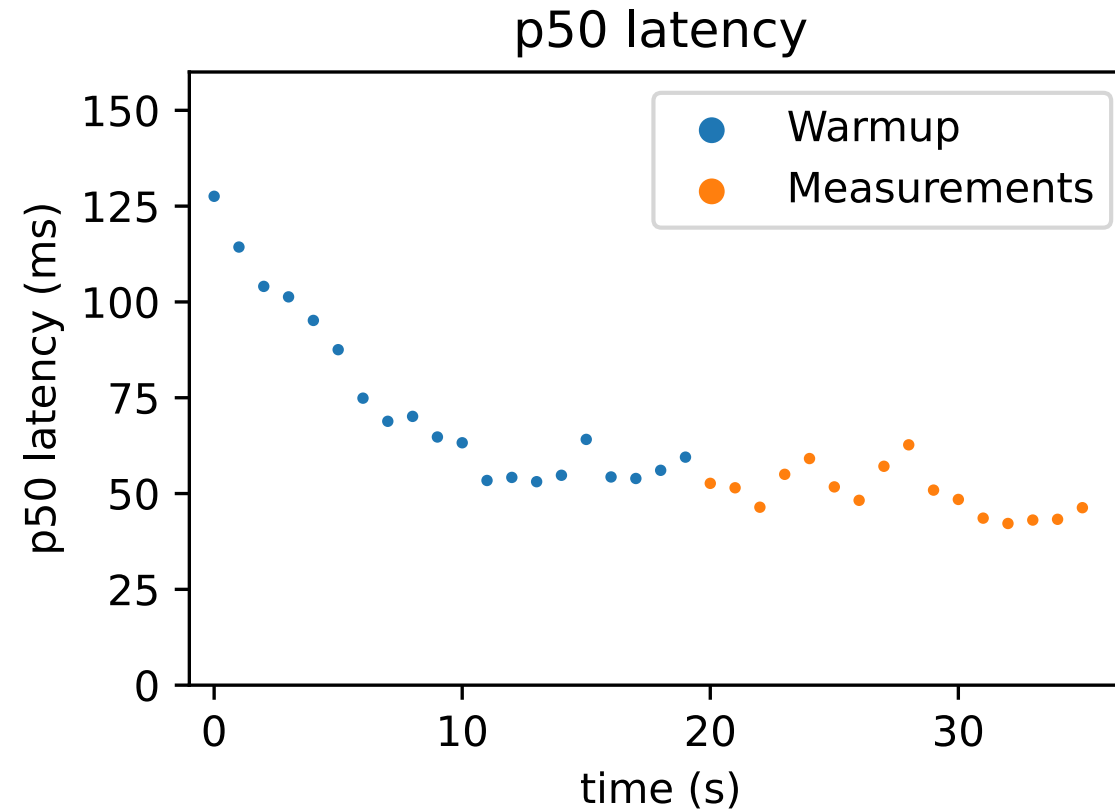
- Goal: Measuring dd-trace-java instrumentation overhead on a Spring app.
- **System under test: Spring app instrumented (or not) with dd-trace-java.**

## An unhappy benchmark

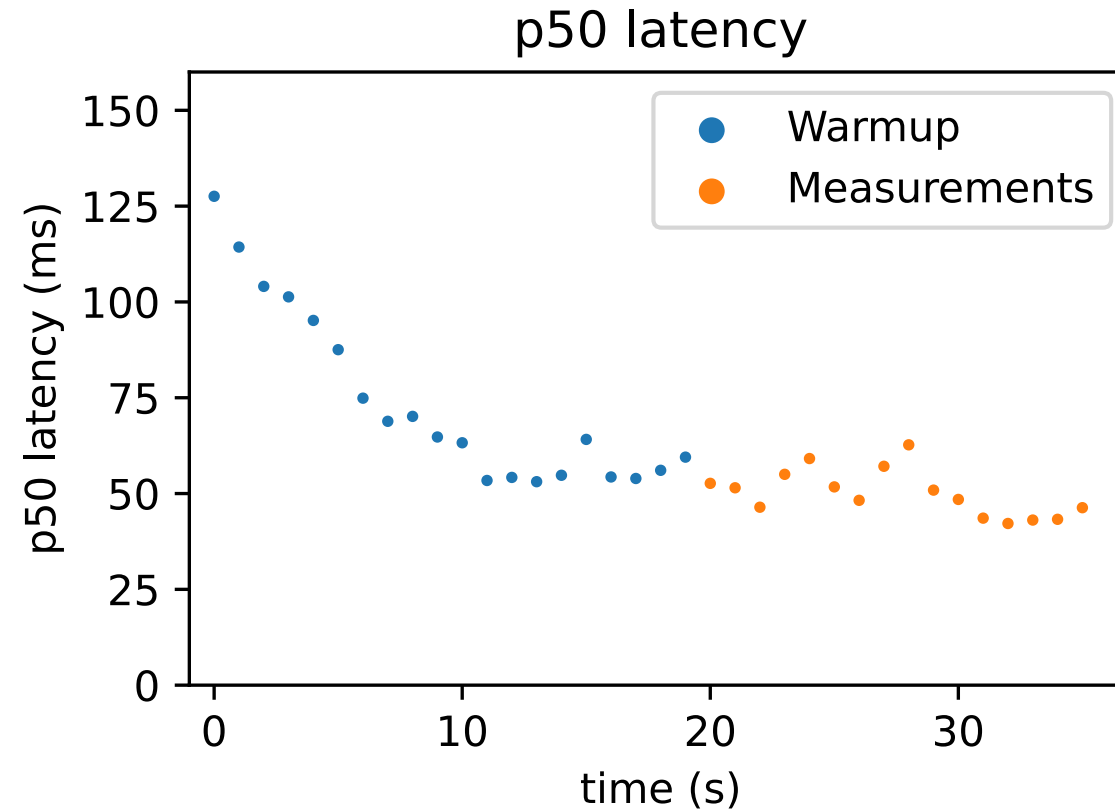
- Goal: Measuring dd-trace-java instrumentation overhead on a Spring app.
- System under test: Spring app instrumented (or not) with dd-trace-java.
- **Workload: As many requests as possible by 5 concurrent users.**

## An unhappy benchmark

- Goal: Measuring dd-trace-java instrumentation overhead on a Spring app.
- System under test: Spring app instrumented (or not) with dd-trace-java.
- Workload: As many requests as possible by 5 concurrent users.
- **20 second warmup, 15 seconds of actual measurements.**

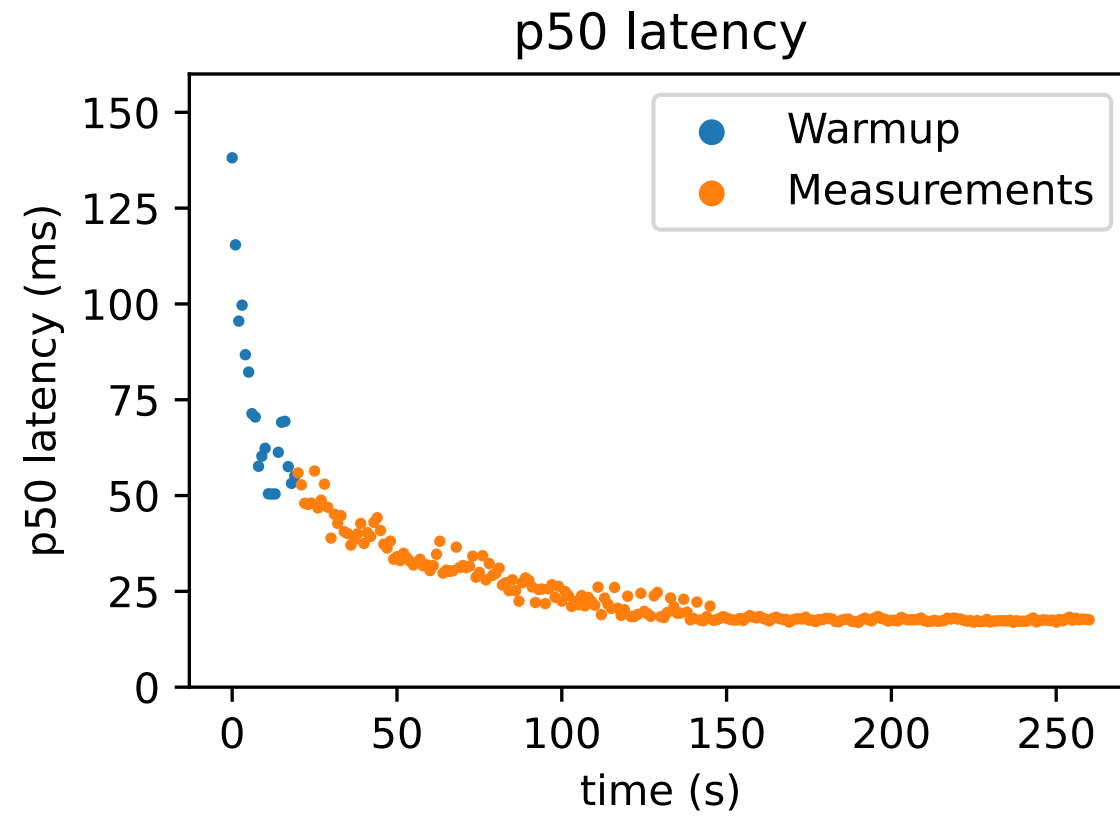


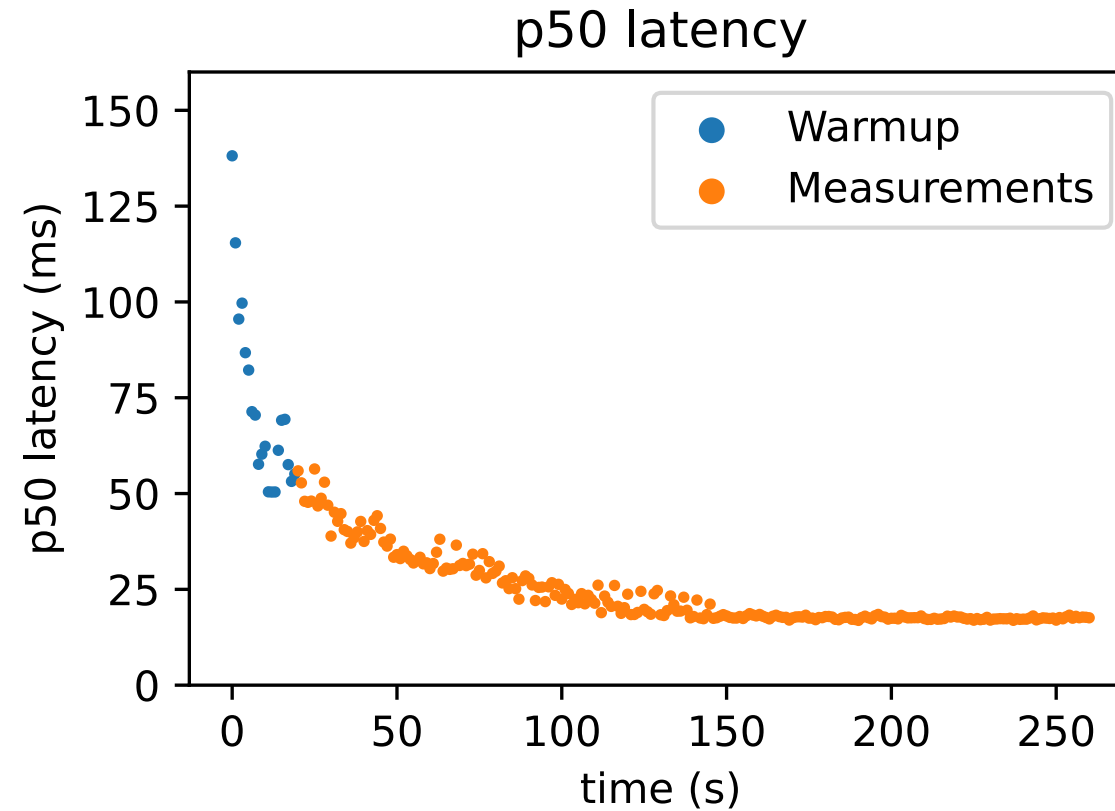
Many **false positives** and **high coeff. of variation** (= standard deviation / mean) of 11.80%.



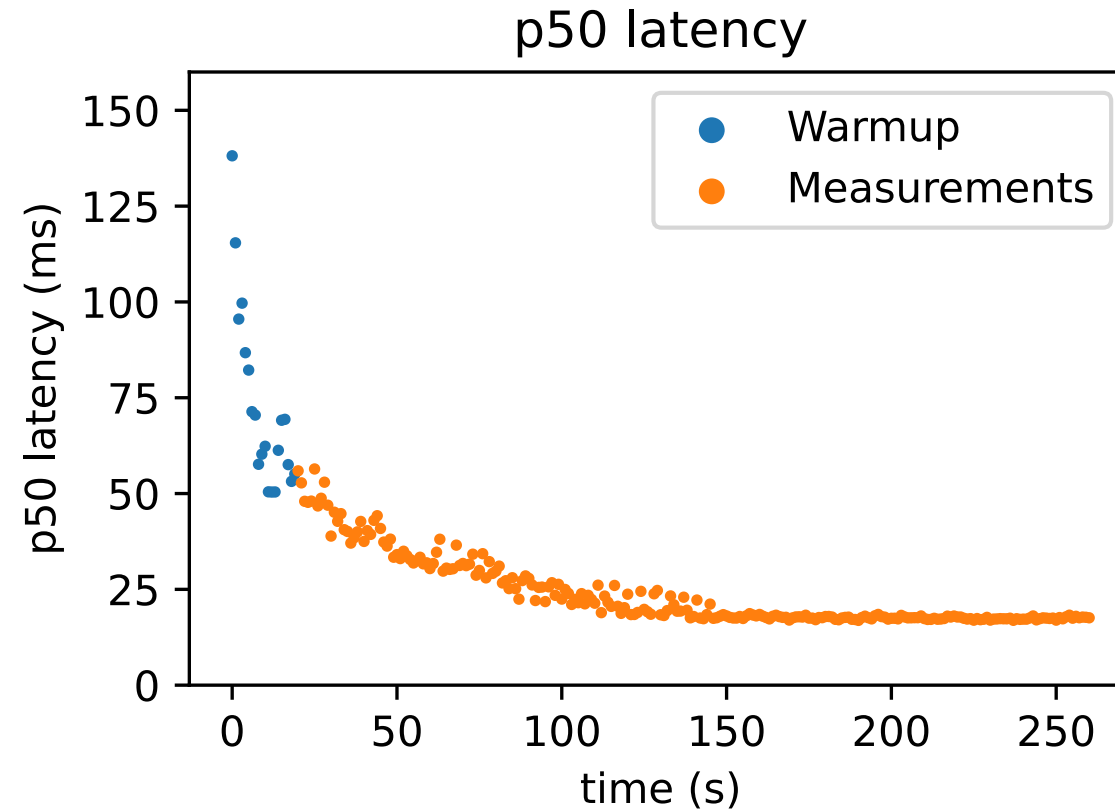
Many **false positives** and **high coeff. of variation** (= standard deviation / mean) of 11.80%.

**Are we running the benchmark long enough?**



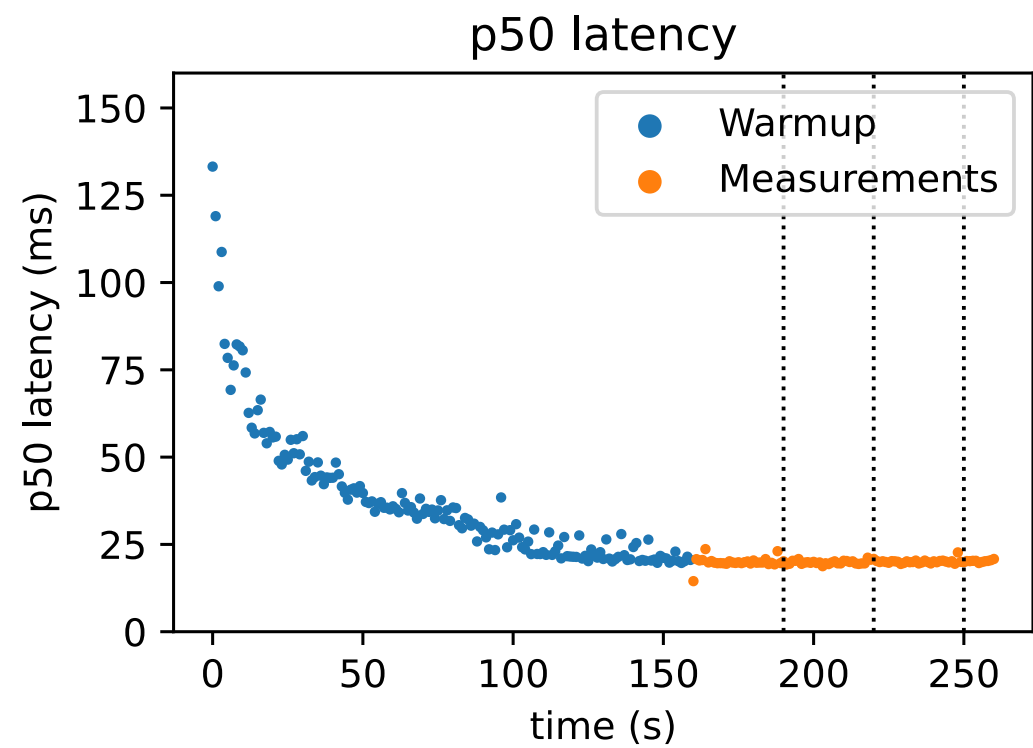


**Tip #1: Run benchmarks for longer to uncover perturbations (e.g., warmup effects).**

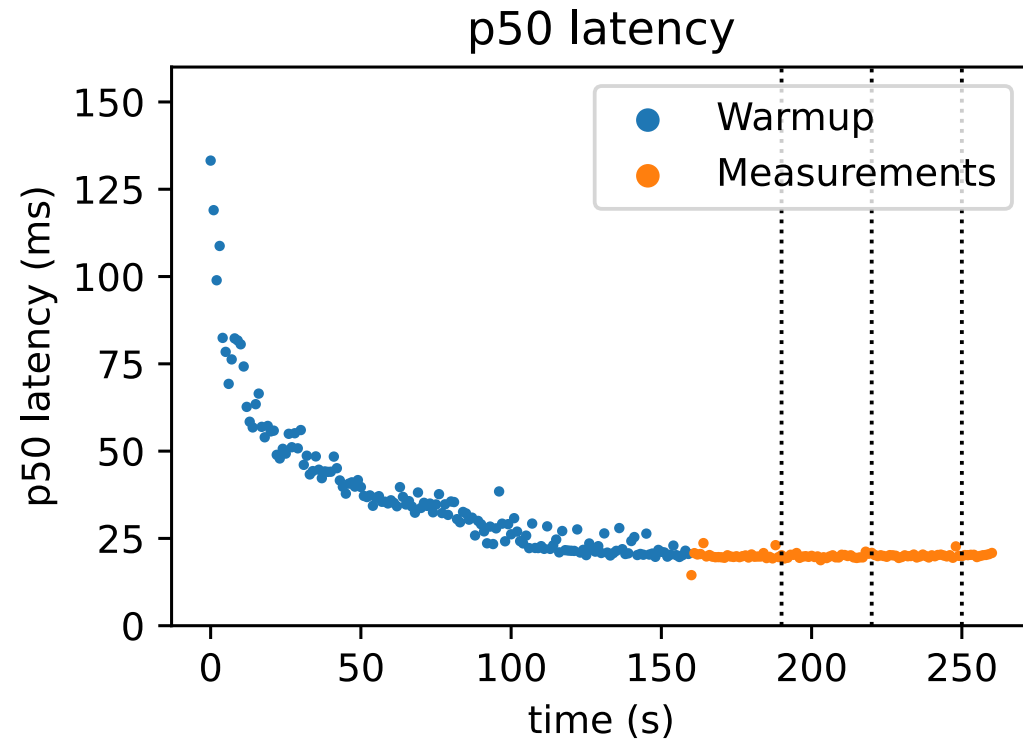


Tip #1: Run benchmarks for longer to uncover perturbations (e.g., warmup effects).

**For how long should we run the benchmark?**

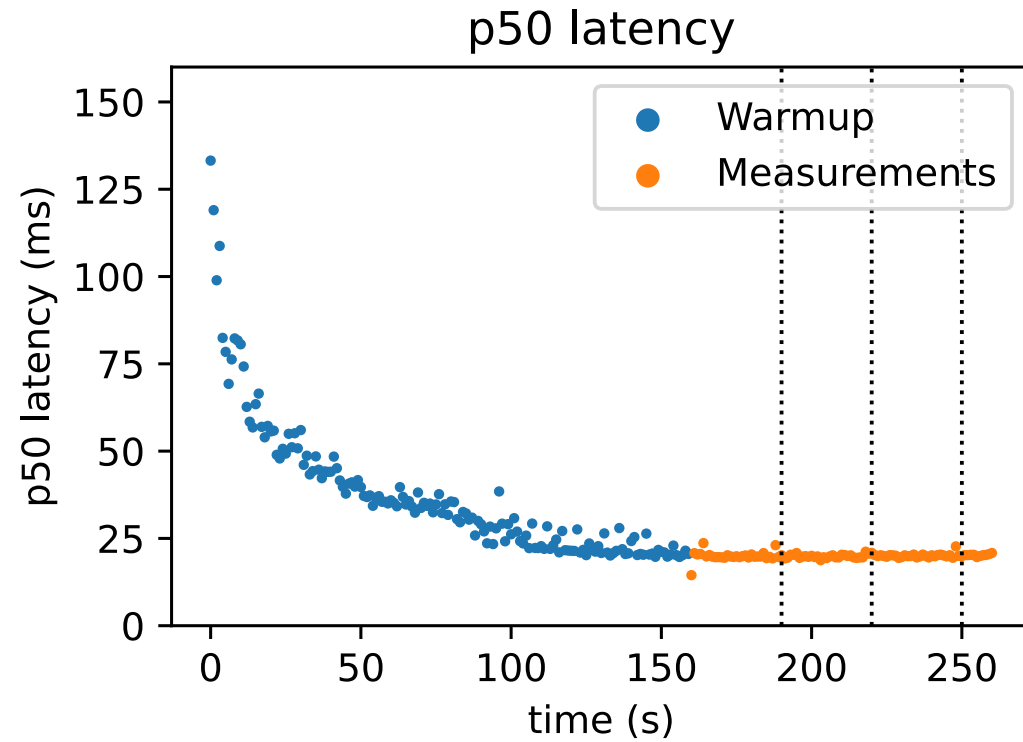


| # measurements | coeff. of variation |
|----------------|---------------------|
| 30             | 6.95%               |
| 60             | 5.23%               |
| 90             | 4.59%               |



| # measurements | coeff. of variation |
|----------------|---------------------|
| 30             | 6.95%               |
| 60             | 5.23%               |
| 90             | 4.59%               |

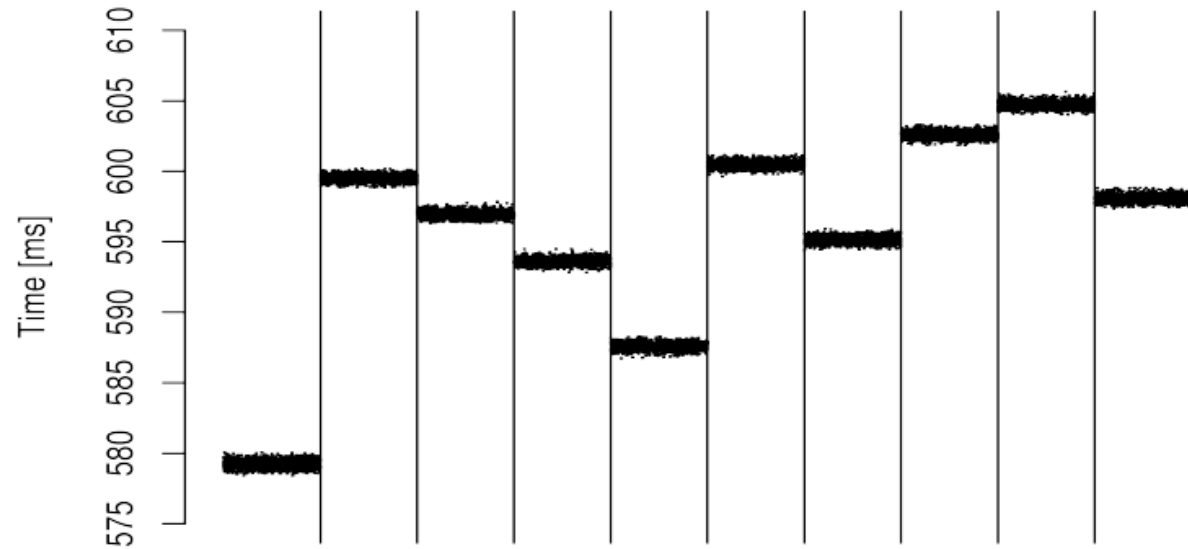
**Tip #2: Collect enough samples to reduce intra-run variation ( $N \geq 30$ ).**



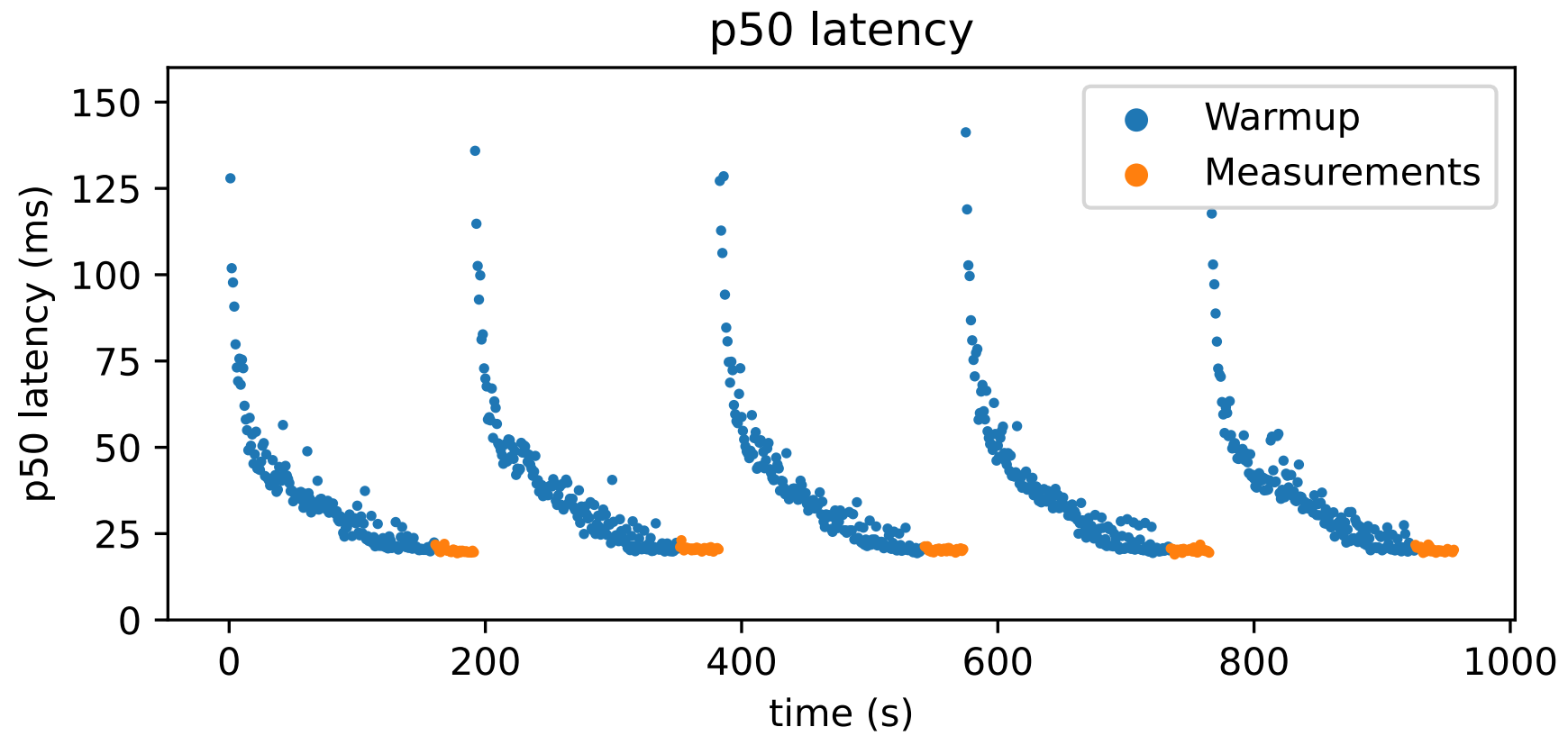
| # measurements | coeff. of variation |
|----------------|---------------------|
| 30             | 6.95%               |
| 60             | 5.23%               |
| 90             | 4.59%               |

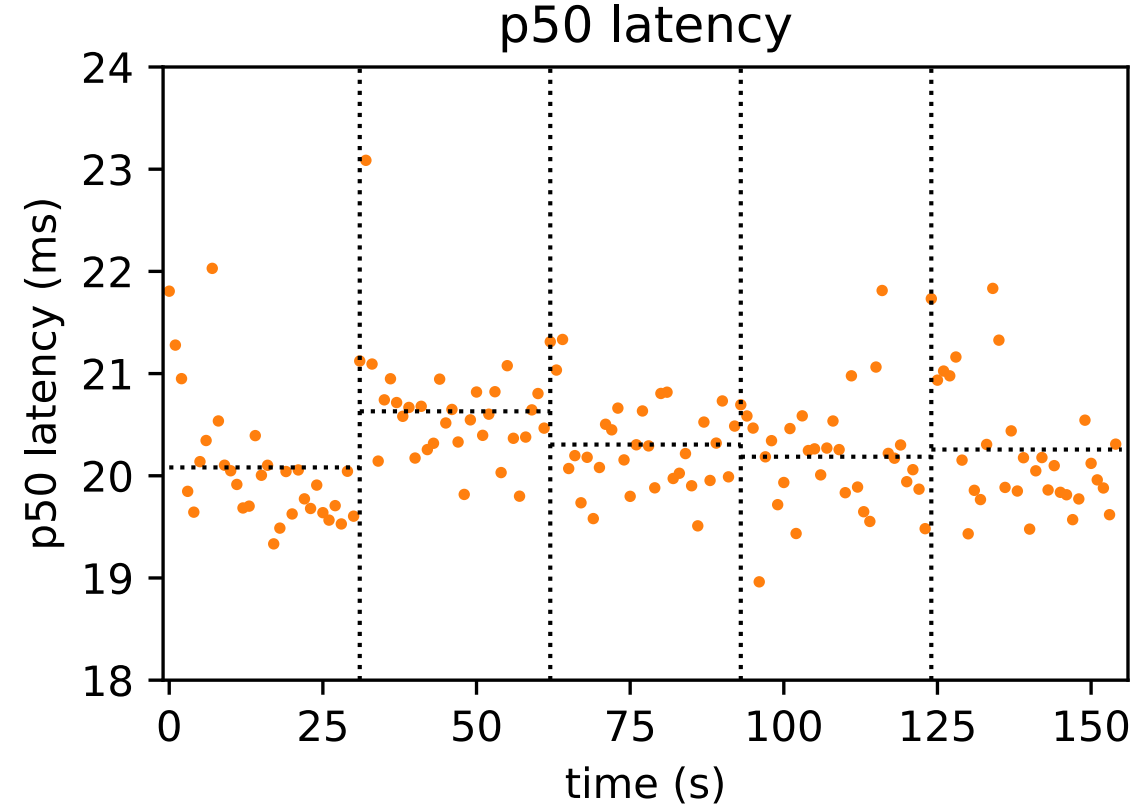
Tip #2: Collect enough samples to reduce intra-run variation ( $N \geq 30$ ).

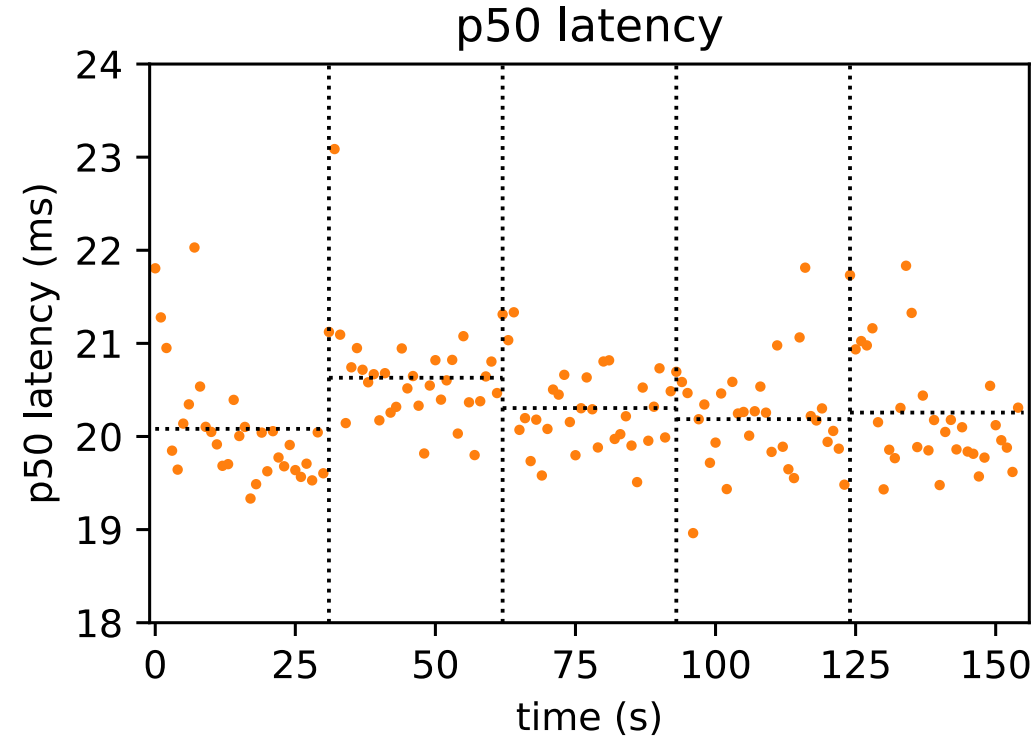
**But what about inter-run variation?**



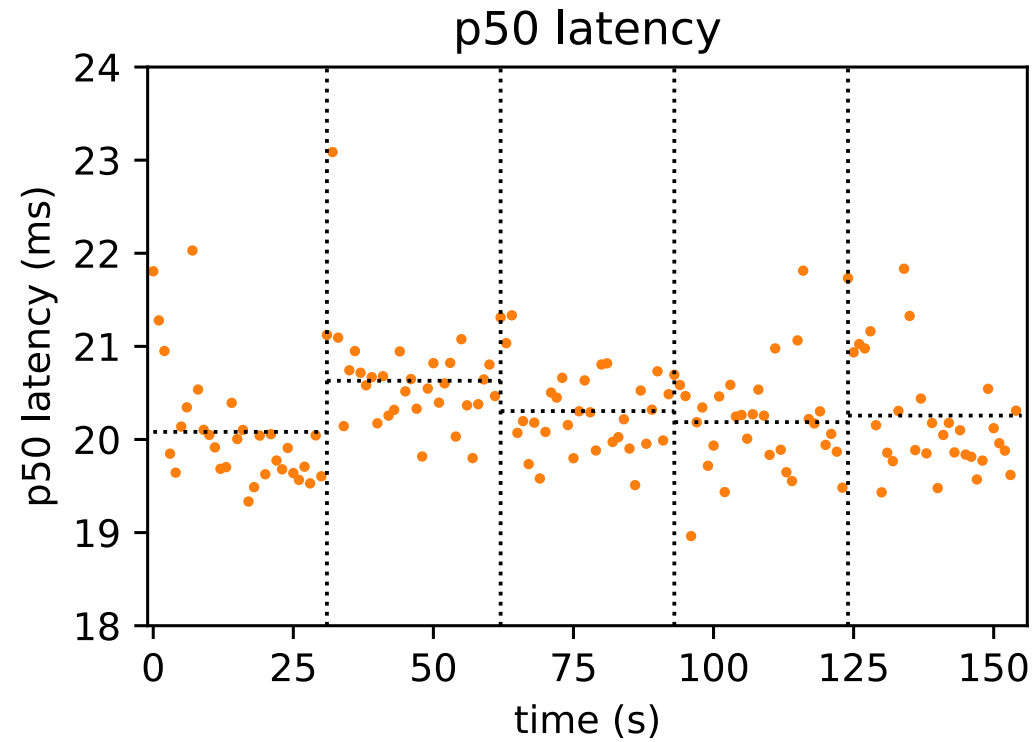
*Impact of initial state on FFT benchmark results [6]*







| Run # | mean ± stddev   | coeff. of variation |
|-------|-----------------|---------------------|
| 1     | 20.08 ± 0.63 ms | 3.16%               |
| 2     | 20.63 ± 0.56 ms | 2.72%               |
| 3     | 20.31 ± 0.45 ms | 2.23%               |
| 4     | 20.19 ± 0.54 ms | 2.66%               |
| 5     | 20.26 ± 0.63 ms | 3.11%               |
| all   | 20.29 ± 0.60 ms | 2.94%               |



| Run # | mean $\pm$ stddev   | coeff. of variation |
|-------|---------------------|---------------------|
| 1     | 20.08 $\pm$ 0.63 ms | 3.16%               |
| 2     | 20.63 $\pm$ 0.56 ms | 2.72%               |
| 3     | 20.31 $\pm$ 0.45 ms | 2.23%               |
| 4     | 20.19 $\pm$ 0.54 ms | 2.66%               |
| 5     | 20.26 $\pm$ 0.63 ms | 3.11%               |
| all   | 20.29 $\pm$ 0.60 ms | 2.94%               |

**Tip #3: Rerun benchmarks multiple times to reduce inter-run variation ( $M \geq 5$ ).**

Tip #1: Run benchmarks for longer to uncover perturbations (e.g., warmup effects).

Tip #2: Collect enough samples to reduce intra-run variation ( $N \geq 30$ ).

Tip #3: Rerun benchmarks multiple times to reduce inter-run variation ( $M \geq 5$ ).

Coefficient of variation: 11.80% → **2.94%**

Tip #1: Run benchmarks for longer to uncover perturbations (e.g., warmup effects).

Tip #2: Collect enough samples to reduce intra-run variation ( $N \geq 30$ ).

Tip #3: Rerun benchmarks multiple times to reduce inter-run variation ( $M \geq 5$ ).

Coefficient of variation: 11.80% → **2.94%**

**Tip #4: Use deterministic inputs.**

Tip #1: Run benchmarks for longer to uncover perturbations (e.g., warmup effects).

Tip #2: Collect enough samples to reduce intra-run variation ( $N \geq 30$ ).

Tip #3: Rerun benchmarks multiple times to reduce inter-run variation ( $M \geq 5$ ).

Coefficient of variation: 11.80% → **2.94%**

Tip #4: Use deterministic inputs.

**Tip #5: Use load generators that avoid the coordinated omission problem (e.g., k6).**

Tip #1: Run benchmarks for longer to uncover perturbations (e.g., warmup effects).

Tip #2: Collect enough samples to reduce intra-run variation ( $N \geq 30$ ).

Tip #3: Rerun benchmarks multiple times to reduce inter-run variation ( $M \geq 5$ ).

Coefficient of variation: 11.80% → **2.94%**

Tip #4: Use deterministic inputs.

Tip #5: Use load generators that avoid the coordinated omission problem (e.g., k6).

**But what about inter-experiment variation?**

Tip #1: Run benchmarks for longer to uncover perturbations (e.g., warmup effects).

Tip #2: Collect enough samples to reduce intra-run variation ( $N \geq 30$ ).

Tip #3: Rerun benchmarks multiple times to reduce inter-run variation ( $M \geq 5$ ).

Coefficient of variation: 11.80% → **2.94%**

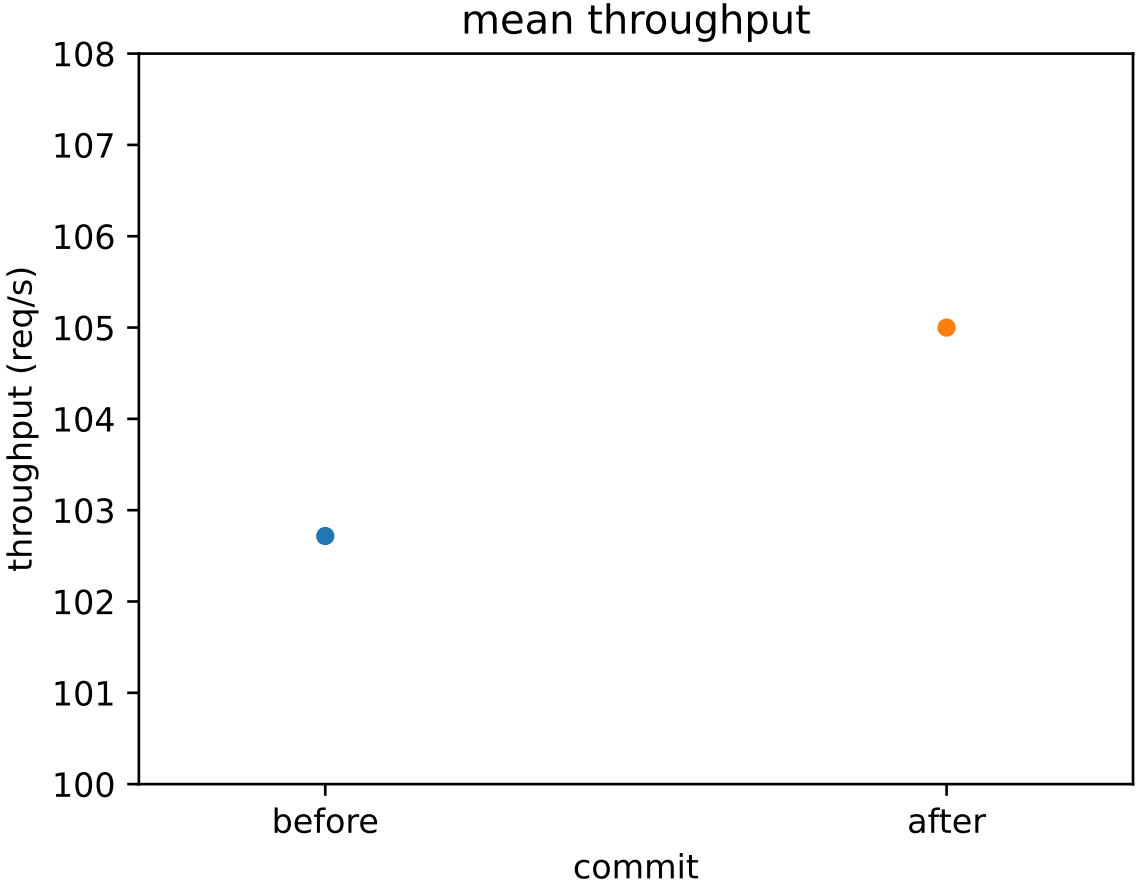
Tip #4: Use deterministic inputs.

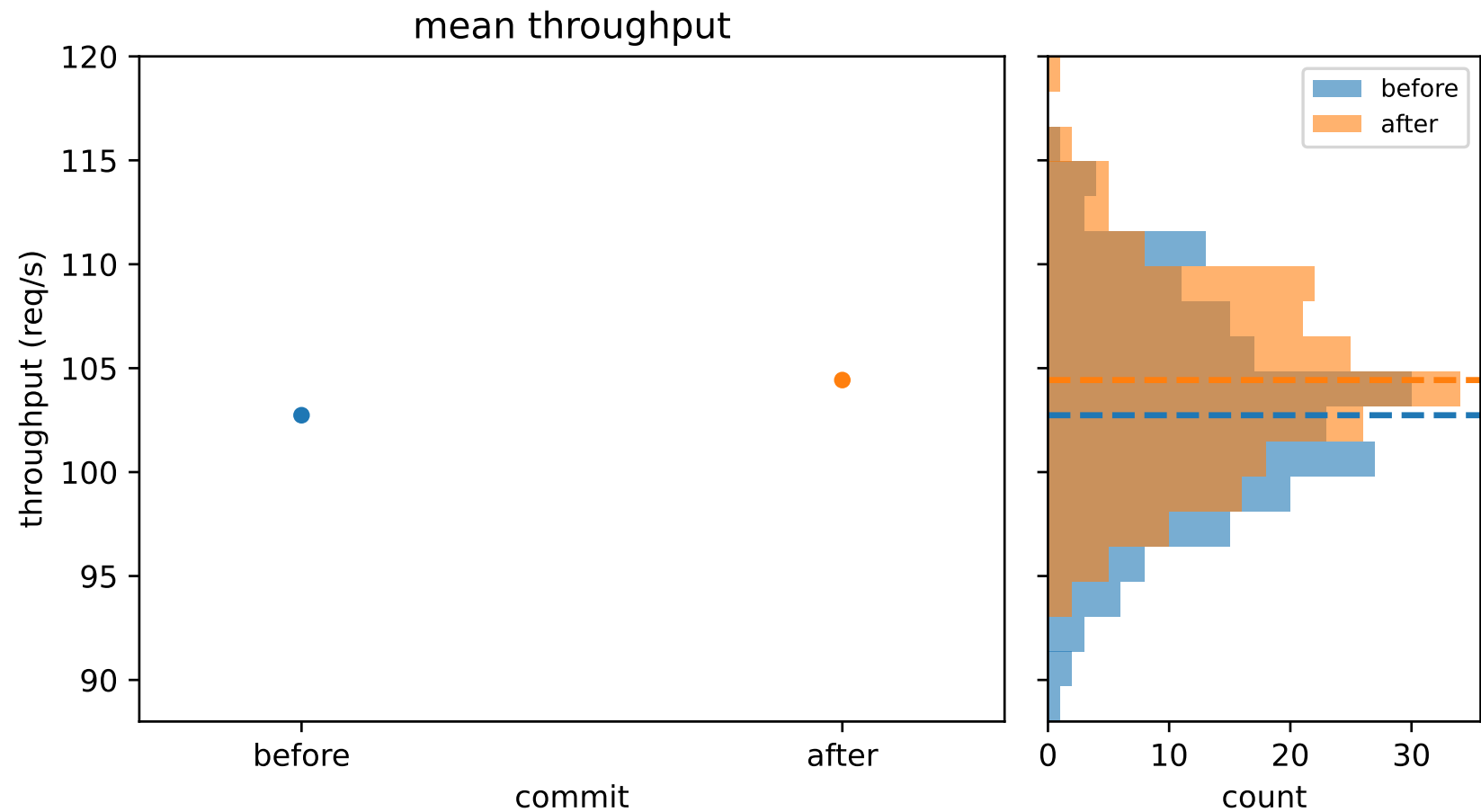
Tip #5: Use load generators that avoid the coordinated omission problem (e.g., k6).

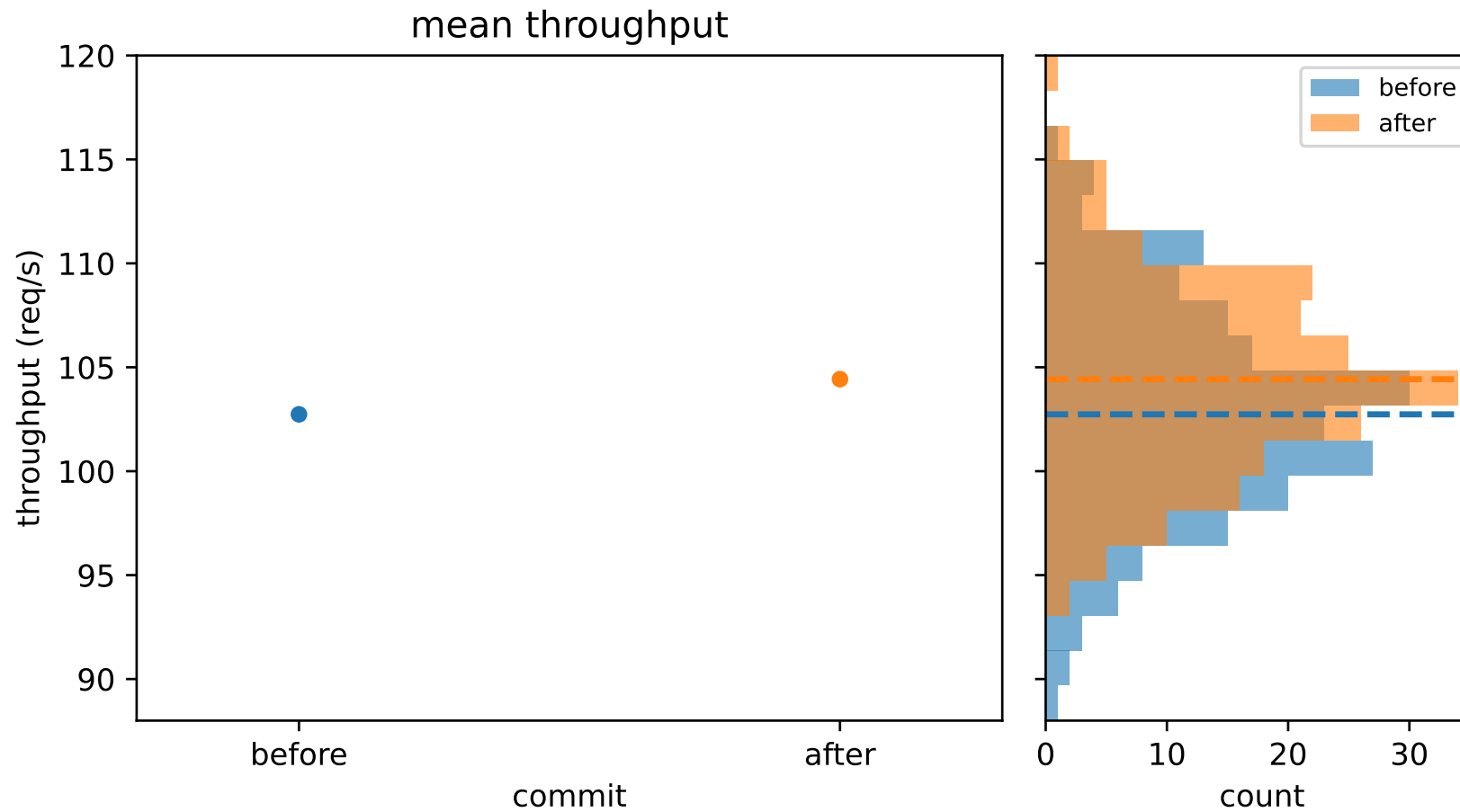
But what about inter-experiment variation?

**Tip #0: Control your benchmarking environment.**

# Interpreting Benchmark Results







**How can we tell if the difference is big enough?**

$$\frac{\text{how big the difference is}}{\text{how big the noise is}}$$

$$t = \frac{\text{how big the difference is}}{\text{how big the noise is}}$$

$$t = \frac{\text{how big the difference is}}{\text{how big the noise is}}$$

$t > \text{critical value}$

$$t = \frac{\text{how big the difference is}}{\text{how big the noise is}} = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$

$$t > \text{critical value}$$

$$t = \frac{\text{how big the difference is}}{\text{how big the noise is}} = \frac{\bar{x}_1 - \bar{x}_2}{\sqrt{\frac{s_1^2}{n_1} + \frac{s_2^2}{n_2}}}$$

$t > \text{critical value}$

**Hypothesis test.**

## Hypothesis test

If  $t > \text{critical value}$ , reject the null hypothesis.

## Hypothesis test

If  $t > \text{critical value}$ , reject the null hypothesis.

**Null hypothesis: no difference.**

## Hypothesis test

If  $t > \text{critical value}$ , reject the null hypothesis.

Null hypothesis: no difference.

**Alternative hypothesis: enough difference.**

## Hypothesis test

If  $t > \text{critical value}$ , reject the null hypothesis.

Null hypothesis: no difference.

Alternative hypothesis: enough difference.

**$t$ , or  $t$ -statistic: difference/noise.**

## Hypothesis test

If  $t > \text{critical value}$ , reject the null hypothesis.

Null hypothesis: no difference.

Alternative hypothesis: enough difference.

$t$ , or  $t$ -statistic: difference/noise.

**Critical value: threshold based on your tolerance for false positives.**

## Hypothesis test

If  $t > \text{critical value}$ , reject the null hypothesis.

Null hypothesis: no difference.

Alternative hypothesis: enough difference.

$t$ , or  $t$ -statistic: difference/noise.

Critical value: threshold based on your tolerance for false positives.

**In practice, we use the  $p$ -value:  $p < \alpha$**

**$p$ -value: probability of seeing this result if there's no real difference.**

**$\alpha$ : false positive rate you're willing to tolerate.**

```
from scipy import stats

alpha = 0.05
t_stat, p_value = stats.ttest_ind(before, after)

if p_value < alpha:
    print("Statistically significant difference")
```

## Choosing $\alpha$

$\alpha = 0.05$  (5%) is a common threshold for false positives.

Confidence level =  $1 - \alpha = 95\%$

## Choosing $\alpha$

$\alpha = 0.05$  (5%) is a common threshold for false positives.

Confidence level =  $1 - \alpha = 95\%$

### Trade-off

- **Lower  $\alpha$  (1%):** Fewer false positives, fewer detections.
- **Higher  $\alpha$  (10%):** More false positives, more detections.

# Another approach: changepoint detection

| Event                                                                                              | Speakers                           | Start | End   |
|----------------------------------------------------------------------------------------------------|------------------------------------|-------|-------|
| <b>Sunday</b>                                                                                      |                                    |       |       |
| Accessible software performance                                                                    | Alexander Zaitsev                  | 09:00 | 09:50 |
| Beyond nvidia-smi: Tools for Real GPU Performance Metrics                                          | YASH PANCHAL                       | 09:50 | 10:30 |
| Keeping the P in HPC: the EESSI Way                                                                | Kenneth Hoste                      | 10:30 | 11:10 |
| Towards unified full-stack performance analysis and automated computer system design with Adaptyst | Maks Graczyk                       | 11:10 | 11:50 |
| <b>How to Reliably Measure Software Performance</b>                                                | Kemal Akkoyun, Augusto de Oliveira | 11:50 | 12:30 |
| Pulling 100k revisions 100× faster                                                                 | Raphaël Gomès, Pierre-Yves David   | 12:30 | 13:10 |
| Database benchmarks: Lessons learned from running a benchmark standard organization                | Gábor Szárnyas                     | 13:10 | 13:50 |
| <b>Continuous Performance Engineering HowTo</b>                                                    | Henrik Ingo                        | 13:50 | 14:30 |
| Writing an ultrafast Lua/JSON encoder+decoder as a LuaJIT module                                   | Adam Ivora                         | 14:30 | 15:10 |
| How To Move Bytes Around                                                                           | Alexey Milovidov                   | 15:10 | 15:50 |
| A Performance Comparison of Kubernetes Multi-Cluster Networking                                    | josecastillolema, Raul             | 15:50 | 16:30 |
| Load Testing Real React Applications for Production Performance                                    | Mohammed Zubair Ahmed              | 16:30 | 17:00 |

# Integrating Benchmarks Into Your Workflows

A series of screenshots showing the different ways in which we integrate benchmarks into our workflows at Datadog, including: a basic architecture slide, reporting capabilities, PR comments, performance quality gates, operational excellence reviews, etc.

## Concluding slides

Summarize the takeaways.

# References

- [1] Universität Münster. "Neutrino oscillations in the neutrino beam from CERN to Gran Sasso." <https://www.uni-muenster.de/Physik.KP/en/AGFrekers/forschung/opera.html>. Accessed Jan 2026.
- [2] CERN. (1999). "From Geneva to Gran Sasso in 2.5 milliseconds!". <https://home.cern/news/press-release/cern/geneva-gran-sasso-25-milliseconds>. Accessed Jan 2026.
- [3] Wikipedia. "OPERA experiment." [https://en.wikipedia.org/wiki/OPERA\\_experiment](https://en.wikipedia.org/wiki/OPERA_experiment). Accessed Jan 2026.
- [4] Strassler, M. (2012). "OPERA: What Went Wrong." <https://profmattstrassler.com/articles-and-posts/particle-physics-basics/neutrinos/neutrinos-faster-than-light/opera-what-went-wrong/>. Accessed Jan 2026.
- [5] Bakhvalov, D. (2020). *Performance Analysis and Tuning on Modern CPUs*.
- [6] Kalibera, T., Bulej, L., and Tuma, P. (2005). "Benchmark Precision and Random Initial State." In *Proceedings of the International Symposium on Performance Evaluation of Computer and Telecommunication Systems (SPECTS)*, pages 182-196. SCS.
- [7] Valles, A. (2009). "Performance Insights to Intel Hyper-Threading Technology." <https://web.archive.org/web/20150217050949/https://software.intel.com/en-us/articles/performance-insights-to-intel-hyper-threading-technology/>. Accessed Jan 2026.
- [8] Gregg, B. (2014). "Frequency Trails: Outliers." <https://www.brendangregg.com/FrequencyTrails/outliers.html#Causes>. Accessed Jan 2026.
- [9] Gregg, B. (2020). "Systems Performance: Enterprise and the Cloud.", p. 233, "P-states and C-states."
- [10] Humenay, E., Tarjan, D., and Skadron, K. (2007). "Impact of Process Variations on Multicore Performance Symmetry."
- [11] Linux Kernel Documentation. "CPUFreq Governors." <https://www.kernel.org/doc/Documentation/cpu-freq/governors.txt>. Accessed Jan 2026.
- [12] ArchWiki. "CPU frequency scaling." [https://wiki.archlinux.org/title/CPU\\_frequency\\_scaling](https://wiki.archlinux.org/title/CPU_frequency_scaling). Accessed Jan 2026.
- [13] Intel. "Intel Server Board and System Products Update on Intel Turbo Boost Technology Support with Low Power Intel Xeon Processor 3400/5500/5600 Series." [https://cdrdv2-public.intel.com/840590/white\\_paper\\_turbo\\_boost\\_on\\_low\\_power\\_processor.pdf](https://cdrdv2-public.intel.com/840590/white_paper_turbo_boost_on_low_power_processor.pdf). Accessed Jan 2026.